#HHMMHHMM

kosu_utils.py

-
- IntegerField CharField BooleanField DateField ForeignKey TextField JSONField
- member Business_Time_

- `on_delete=SET_NULL`
-

-
-
-

`models.py`

-
-
- `employee_no`
-
- `python manage.py makemigrations` `migrate`

```
#
# Django
from django.db import models

# models.Model
class Member(models.Model):
    #
    employee_no = models.IntegerField('          ')
    #     100
    name = models.CharField('    ', max_length=100)
```

- `class Member(models.Model):` `models.Model`

- `models.IntegerField(...)`
- `models.CharField(..., max_length=100)`
  `max_length` `CharField`

`__str__`

`def __str__(self):`

```
#
def __str__(self):
    #
    return self.name
```

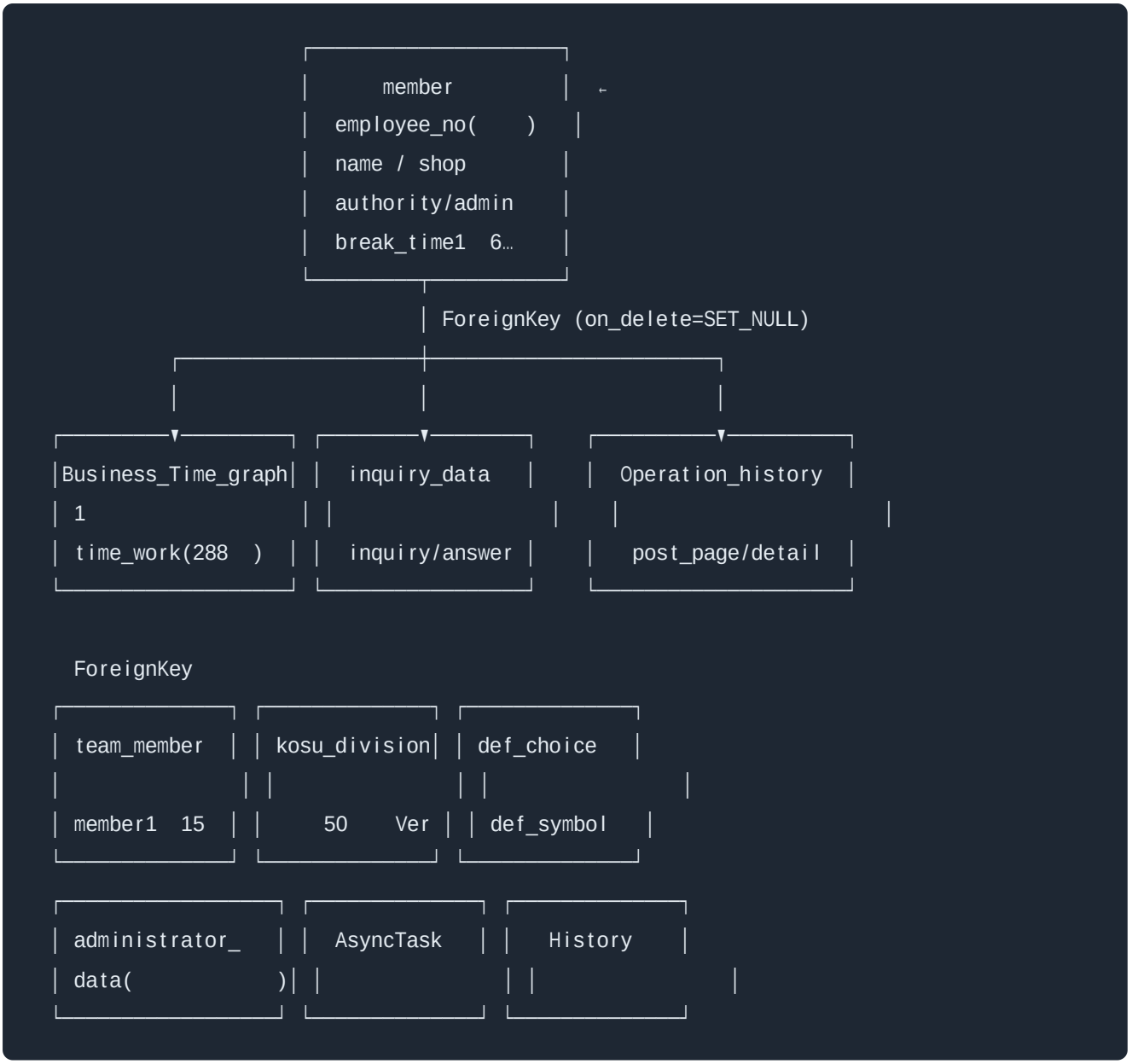
- `self`
- `return self.name` `member object (1)`

`IntegerField`				`employee_no`
`CharField`		`max_length`		`name` `time_work`
`TextField`				`inquiry`
`BooleanField`				`authority` `follow`
`DateField`				`work_day2`
`DateTimeField`				`created_at`
`ForeignKey`				`name`
`JSONField`				`changes`

> **null** **blank** ⚠️
>
> - `null=True`
>
> - `blank=True`
> `CharField` `null=True`
> `''`

> **verbose_name** `models.IntegerField('')`
> `''`
> `employee_no`

>



- **member** ForeignKey member
- Business_Time_graph inquiry_data Operation_history name member
- team_member kosu_division def_choice administrator_data AsyncTask History member ForeignKey

---

models.py

```
#
class member(models.Model):
  #                          :DB      ,    :
  shop_list = [
    #         P
    ('P', 'P'),
    #         R
    ('R', 'R'),
    #         W1
    ('W1', 'W1'),
    #         W2
    ('W2', 'W2'),
    #         T1
    ('T1', 'T1'),
    #         T2
    ('T2', 'T2'),
    #         A1
    ('A1', 'A1'),
    #         A2
    ('A2', 'A2'),
    #         J
    ('J', 'J'),
    #
    ('      ', '      '),
    #         B
    ('        (P,R,T,      )', '        (P,R,T,      )'),
    #         C
    ('        (W,A)', '        (W,A)'),
    #
    ('          ', '          '),
  ]
```

- `shop_list = [ ... ]` `(      ,      )`
- `('P', 'P')`

- `'          '`

`shop = models.CharField(..., choices=shop_list, ...)` `choices`

⚠️ `shop` `kosu_utils.py` `judgement_check` `kosu_sort`

```
#
employee_no = models.IntegerField('          ')
#          100
name = models.CharField('    ', max_length=100)
#       §4.1
shop = models.CharField('        ', choices = shop_list, max_length=15)
#                 True/False
authority = models.BooleanField('    ')
#              True/False
administrator = models.BooleanField('      ')
```

- `employee_no`
- `name`
- `shop` `choices=shop_list` `max_length=15` `'       (P,R,T,      )'`
- `authority` `True` `False`
- `administrator` `True`

> `True` `False`

> `administrator=False` `authority=True` `authority=False,` `administrator=True`

`member`

> `#HHMMHHMM`

```
# 1
break_time1 = models.CharField('1           ', max_length=9)
# 1           1
break_time1_over1 = models.CharField('1             1', max_length=9)
# 1           2
break_time1_over2 = models.CharField('1             2', max_length=9)
# 1           3
break_time1_over3 = models.CharField('1             3', max_length=9)
# 2
break_time2 = models.CharField('2           ', max_length=9)
# 2           1
break_time2_over1 = models.CharField('2             1', max_length=9)
# 2           2
break_time2_over2 = models.CharField('2             2', max_length=9)
# 2           3
break_time2_over3 = models.CharField('2             3', max_length=9)
# 3
break_time3 = models.CharField('3           ', max_length=9)
# 3           1
break_time3_over1 = models.CharField('3             1', max_length=9)
# 3           2
break_time3_over2 = models.CharField('3             2', max_length=9)
# 3           3
break_time3_over3 = models.CharField('3             3', max_length=9)
#
break_time4 = models.CharField('             ', max_length=9)
#              1
break_time4_over1 = models.CharField('               1', max_length=9)
#              2
break_time4_over2 = models.CharField('               2', max_length=9)
#              3
break_time4_over3 = models.CharField('               3', max_length=9)
#   1
break_time5 = models.CharField('  1           ', max_length=9)
break_time5_over1 = models.CharField('  1             1', max_length=9)#   1
    1
break_time5_over2 = models.CharField('  1             2', max_length=9)#   1
    2
break_time5_over3 = models.CharField('  1             3', max_length=9)#   1
    3
#   2
break_time6 = models.CharField('  2           ', max_length=9)
break_time6_over1 = models.CharField('  2             1', max_length=9)#   2
    1
break_time6_over2 = models.CharField('  2             2', max_length=9)#   2
    2
```

```
    break_time6_over3 = models.CharField('  2              3', max_length=9)#   2
        3
```

`break_time1`			
`break_time2`			
`break_time3`			
`break_time4`			`tyoku=='4'`
`break_time5`			`tyoku=='5'`
`break_time6`			`tyoku=='6'`

> ⚠️ **`max_length=9`** `#HHMMHHMM` `#`

> `break_get` `tyoku`

`kosu_utils.py` `break_get`

> `tyoku`

```
#
#     :   ,
def break_get(tyoku, login_no):
  #
  #       1
  break_time_obj = member.objects.get(employee_no=login_no)

  # 1
  #     1
  if tyoku == '1':
    # 1
    breaktime = break_time_obj.break_time1
    # 1            1
    breaktime_over1 = break_time_obj.break_time1_over1
    # 1            2
    breaktime_over2 = break_time_obj.break_time1_over2
    # 1            3
    breaktime_over3 = break_time_obj.break_time1_over3
  #   2      2               break_time2  break_time6
  ...
  # 4
  return breaktime, breaktime_over1, breaktime_over2, breaktime_over3
```

- `member.objects.get(employee_no=login_no)`
- `if tyoku == '1':` `break_time2` `break_time6`
-

```
  #            1
  pop_up1 = models.CharField('           1', max_length=255, null=True, blank=True)
  #         ID1
  pop_up_id1 =  models.CharField('           ID1', max_length=255, null=True,
blank=True)
  #            2
  pop_up2 = models.CharField('           2', max_length=255, null=True, blank=True)
  #         ID2
  pop_up_id2 =  models.CharField('           ID2', max_length=255, null=True,
blank=True)
  #            3
  pop_up3 = models.CharField('           3', max_length=255, null=True, blank=True)
  #         ID3
  pop_up_id3 =  models.CharField('           ID3', max_length=255, null=True,
blank=True)
  #            4
  pop_up4 = models.CharField('           4', max_length=255, null=True, blank=True)
  #         ID4
  pop_up_id4 =  models.CharField('           ID4', max_length=255, null=True,
blank=True)
  #            5
  pop_up5 = models.CharField('           5', max_length=255, null=True, blank=True)
  #         ID5
  pop_up_id5 =  models.CharField('           ID5', max_length=255, null=True,
blank=True)
```

- `null=True, blank=True`
- `pop_up1` `pop_up_id1`

`kosu_utils.py` `break_time_delete`

```
#
break_check = models.BooleanField('                    ', null=True)

#
def __str__(self):
  #
  return self.name
```

- `break_check == True`
- `break_check == False` `None`

`time_work`

```
#
class Business_Time_graph(models.Model):
  #
  employee_no3 = models.IntegerField('          ')
  # member
  name = models.ForeignKey(member, verbose_name='    ', null=True,
on_delete=models.SET_NULL)
  #
  def_ver2 = models.CharField('           Ver', max_length=100, blank=True,
null=True)
  #
  work_day2 = models.DateField('      ')
  #     '1'  '6'
  tyoku2 = models.CharField('  ', max_length=2, blank=True, null=True)
  #   1  288
  time_work = models.CharField('        ', max_length=288, blank=True, null=True)
  #
  detail_work = models.CharField('        ', max_length=32676, blank=True, null=True)
  #
  over_time = models.IntegerField('        ', blank=True, null=True)
  #                  #HHMMHHMM
  breaktime = models.CharField('          ', max_length=9, blank=True, null=True)
  #         1
  breaktime_over1 = models.CharField('           1', max_length=9, blank=True,
null=True)
  #         2
  breaktime_over2 = models.CharField('           2', max_length=9, blank=True,
null=True)
  #         3
  breaktime_over3 = models.CharField('           3', max_length=9, blank=True,
null=True)
  #     /    /
  work_time = models.CharField('        ', max_length=100, blank=True, null=True)
  #       OK    §14
  judgement = models.BooleanField('        OK_NG', null=True)
  #
  break_change = models.BooleanField('               ', null=True)

  #
  def __str__(self):
    #   id__    :
    return str(self.id) + '__' + str(self.work_day2) + ':' + str(self.employee_no3)
```

- `employee_no3` `member` `employee_no` `name` `SET_NULL`

- `name = models.ForeignKey(member, ...)` **member**
  `on_delete=models.SET_NULL` `null=True`

- `def_ver2`
  `kosu_division.kosu_name`

- `work_day2` `DateField`
- `tyoku2` `'1'` `'6'`
  `kosu_utils.py`
- **time_work** `max_length=288`

- **detail_work** `$` `max_length=32676`

- `over_time` `judgement_check`

- `breaktime` `breaktime_over3` `member`
  `#HHMMHHMM`
- `work_time` `'  '` `'  '` `'  '` `'  '`
  `judgement_check`
- `judgement` `judgement_check`

- `break_change`
  on_delete=SET_NULL

> `name`

```
  name = models.ForeignKey(member, verbose_name='  ', null=True,
on_delete=models.SET_NULL)
```

`on_delete`

`CASCADE`		X
`SET_NULL`	**`name`**	
`PROTECT`		

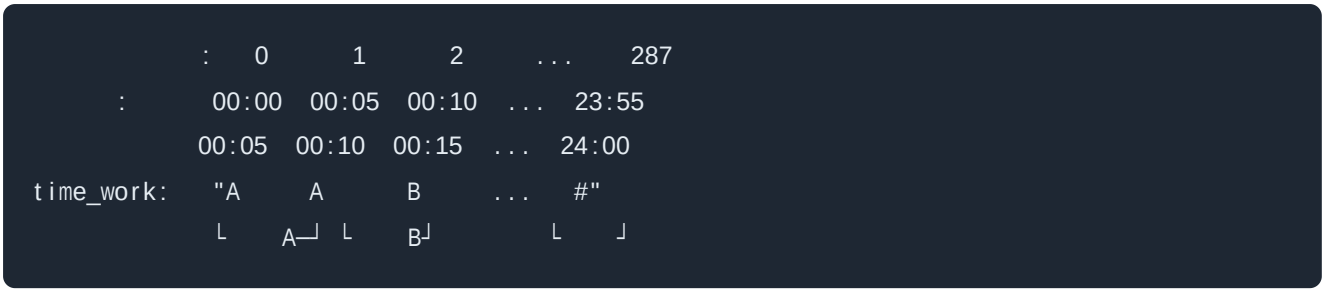
`SET_NULL`

> **`employee_no3`** `name` `NULL`
> `employee_no3`

> ⚠️ `inquiry_data` `Operation_history` `name = ForeignKey(member, ..., on_delete=SET_NULL)`

```
24 × 60 ÷ 5 = 1440 ÷ 5 = 288
```

`time_work` `max_length=288`



> time_work

A Z a x	kosu_division
#	
$	

> A x def_choice def_list kosu_division_dictionary "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwx"

kosu_utils.py



- × 12
- MM ÷ 5

```
12 × 12 + 30 ÷ 5 = 144 + 6 = 150
→ time_work   150      0          12:30-12:35
```

`t` `create_kosu_basic`

```
  = t // 12          12
  = (t % 12) × 5     12              × 5
```

`//` `%`

- `//` `150 // 12 = 12`
- `%` `150 % 12 = 6` `6 × 5 = 30`

`member.break_time1` `Business_Time_graph.breaktime`

`max_length=9` `#12451330`

```
 #   1 2   4 5   1 3   3 0
 |   └─┘   └─┘   └─┘   └─┘
 |    |     |     |     |

(#)    (12)  (45)  (13)  (30)

   :   12  45      13  30
```

	#	
	12	
	45	
	13	
	30	

> # parse_break_time '#' + ...

#23450015

break_time_process

kosu_utils.py #HHMMHHMM

> #

```
def parse_break_time(break_time_start, break_time_end, jst):
  #             : None
  #
  if not break_time_start or not break_time_end:
    #
    raise ValueError("                    ")

  try:
    #       →
    start_time = datetime.datetime.strptime(break_time_start, "%Y-%m-%dT%H:%M:%S.%fZ")
    #       →
    end_time = datetime.datetime.strptime(break_time_end, "%Y-%m-%dT%H:%M:%S.%fZ")
    start_time = (
        # UTC
        start_time.replace(tzinfo=datetime.timezone.utc)
        #         (JST)
        .astimezone(jst)
        # "HH:MM"
        .strftime("%H:%M")
    )
    end_time = (
        #
        end_time.replace(tzinfo=datetime.timezone.utc)
        .astimezone(jst)
        .strftime("%H:%M")
    )
    #   #HHMMHHMM
    time_str = '#' + start_time[0:2] + start_time[3:5] + end_time[0:2] + end_time[3:5]
    # "HH:MM"→  ,    §8
    start_time_hour, start_time_min = time_index(start_time)
    end_time_hour, end_time_min = time_index(end_time)
    #                    §6.3
    start_time_ind = int(int(start_time_hour) * 12 + int(start_time_min) / 5)
    #
    end_time_ind = int(int(end_time_hour) * 12 + int(end_time_min) / 5)
  except ValueError:
    #
    raise ValueError("                      ")

  #     IND,     IND, #HHMMHHMM
  return start_time_ind, end_time_ind, time_str
```

- `time_str = '#' + start_time[0:2] + start_time[3:5] + ...` `start_time` `"12:45"` `[0:2]` `"12"` `[3:5]` `"45"` `:`

`#`

- `start_time_ind = int(int(...) * 12 + int(...) / 5)`
  `int(...)`

> **raise**
> `try / except`

> `"12:45"`
> `:`

```
#
#     : "HH:MM"
def time_index(post_time):
  #
  # ':'                      "12:45"    2
  post_index = post_time.index(':')
  #
  #          ':'             → "12"
  time_hour = post_time[ : post_index]
  #
  # ':'                   → "45"
  time_min = post_time[post_index + 1 : ]

  #
  # ("12", "45")
  return time_hour, time_min
```

- `post_index = post_time.index(':')` `.index(':')` `:` `"12:45"` `2`
- `time_hour = post_time[ : post_index]` `post_index` `"12"`
- `time_min = post_time[post_index + 1 : ]` `post_index+1` `"45"` `:`
- `return time_hour, time_min`

> `[ : ]` `"12:45"[0:2]` `"12"`

> ```
> time_index("09:30")  →  ("09", "30")
> time_index("23:05")  →  ("23", "05")
> ```

> `parse_break_time`

> `#HHMMHHMM`

```
#
#     : "#HHMMHHMM"
def break_time_process(breaktime_str):
  #
  #     HH×12 +     MM÷5
  break_start = int(breaktime_str[1 : 3])*12 + int(breaktime_str[3 : 5])/5
  #
  #     HH×12 +     MM÷5
  break_end = int(breaktime_str[5 : 7])*12 + int(breaktime_str[7 :])/5

  #
  #             0
  break_next_day = 0
  #
  #      >              :23:45→00:15
  if break_start > break_end:
    #               1
    #
    break_next_day = 1

  #
  #
  else:
    #               0
    #
    break_next_day = 0

  #     IND,     IND,
  return break_start, break_end, break_next_day
```

- `breaktime_str[1 : 3]` `#`
- `breaktime_str[3 : 5]`
- `breaktime_str[5 : 7]`
- `breaktime_str[7 : ]`

`HH×12 + MM÷5`

> `"#12451330"`

```
break_start = 12×12 + 45/5 = 144 + 9  = 153.0
break_end   = 13×12 + 30/5 = 156 + 6  = 162.0
153 < 162        break_next_day = 0
```

`"#23450015"`

```
break_start = 23×12 + 45/5 = 276 + 9  = 285.0
break_end   =  0×12 + 15/5 =   0 + 3  =   3.0
285 > 3        break_next_day = 1
```

⚠️

```
#
#       ,       ,        List,    List,
def kosu_write(start_ind, end_ind, kosu_def, detail_list, post_data):
  #
  #
  for kosu in range(start_ind, end_ind):
    #
    #
    kosu_def[kosu] = post_data.get('time_work')
    #
    #
    detail_list[kosu] = post_data.get('detail_work')

  #          2
  return kosu_def, detail_list
```

- `for kosu in range(start_ind, end_ind):` `range(  ,  )`
  `range(150, 153)` `150, 151, 152`

- `kosu_def[kosu] = post_data.get('time_work')` `kosu`
  `A`
- `detail_list[kosu] = post_data.get('detail_work')`

- `post_data.get('time_work')` `post_data`
  `'time_work'` `.get()` `None`

> `kosu_def[3]`
> `time_work` `list(...)`
> `kosu_sort` `list(obj_get.time_work)`

> `kosu_write(150, 153, kosu_def, detail_list,`
> `{'time_work':'A','detail_work':'  '})` `kosu_def[150], kosu_def[151],`
> `kosu_def[152]` `'A'` `'  '`

`break_check`

```
#
#     ,     ,    List,    List,
def break_time_delete(break_start_ind, break_end_ind, work_list, detail_list,
member_obj):
  #
  #
  for bt in range(int(break_start_ind), int(break_end_ind)):
    #                                 ON
    #
    if member_obj.break_check == True:
      #
      #             (#)
      if work_list[bt] != '#':
        #
        #     ($)          =
        if work_list[bt] != '$':
          #
          return '                                          or
        ', None, None

    #                                 OFF
    #
    else:
      #
      #             (#)
      work_list[bt] = '#'
      #
      #
      detail_list[bt] = ''

  #           ,         2
  return None, work_list, detail_list
```

- `for bt in range(int(break_start_ind), int(break_end_ind)):` `int(...)` `153.0`
- `if member_obj.break_check == True:` `member_obj` `break_check`
  - `work_list[bt] != '#'` `!= '$'` `return ', None, None`
  - `work_list[bt] = '#'` `detail_list[bt] = ''`
- `return None, work_list, detail_list` `None`

> `None` `None`

> ⚠️ **`break_check`**
> `break_check=True` `False`

`$` `break_time_delete`

> `$`

```
#
#       ,       ,    List,    List
def break_time_write(break_start_ind, break_end_ind, kosu_def, detail_list):
  #
  #
  for bt in range(int(break_start_ind), int(break_end_ind)):
    #
    #                ($)
    kosu_def[bt] = '$'
    #
    detail_list[bt] = ''

  #         2
  return kosu_def, detail_list
```

- `kosu_def[bt] = '$'` `$`
- `detail_list[bt] = ''`

`break_time_delete`

- `break_time_delete` `'#'`

- `break_time_write` `'$'`
  `#` `$`
  `$` `#`

`$` `judgement_check` `kosu_def.count('$')*5`
`$` `$`

`detail_work`

$

$

```
#
#     : 288
def detail_list_summarize(detail_list):
  #         str
  #
  detail_list_str = ''

  #                 str
  #             i
  for i, e in enumerate(detail_list):
    #
    #
    if i == len(detail_list) - 1:
      #                                     str
      #                           $
      detail_list_str = detail_list_str + detail_list[i]

    #
    #
    else:
      #                                     str                         '$'
      #      +       '$'
      detail_list_str = detail_list_str + detail_list[i] + '$'

  #         1
  return detail_list_str
```

- `enumerate(detail_list)` `i` `e` `i`
- `if i == len(detail_list) - 1:` `len(...)` `288 - 1 = 287`
- `detail_list_str + detail_list[i]` `$`
- `detail_list_str + detail_list[i] + '$'` `$`

`$` `kosu_sort`
`obj_get.detail_work.split('$')` `A$B$C` `$` `['A','B','C']`

> $ ['A','B','C','']

> ⚠️ $ $ time_work
> detail_work

Business_Time_graph.judgement

> judgement=True

```
#
#    List,       ,  ,    ,
def judgement_check(kosu_def, work, tyoku, member_obj, over_work):
  #
  # 1  1440   -       -
  kosu_total = 1440 - (kosu_def.count('#')*5) - (kosu_def.count('$')*5)

  #        OK_NG
  #    NG(False)
  judgement = False
```

- kosu_total = 1440 - (kosu_def.count('#')*5) - (kosu_def.count('$')*5)
  - 1440

- `kosu_def.count('#')` ×5
- `kosu_def.count('$')` ×5
- `kosu_total`
- `judgement = False`

```
#
#
if (work == '    ' or work == '        ') and \
  #       -      == 470  (    ) ÆÉÓÝ¤ÐÑ¤Ù™ÚØ¤Ø  kosu_total
  kosu_total - int(over_work) == 470:
  #         OK_NG  OK    ﻁﻲ
  # OK
  judgement = True

ÐÑ
o ﻁﻲì æ †œ¤ÚU%PÐ
```

>

```
#
#         (4)
if tyoku == '4':
  #
  #          →    230
  if work == '         ' and kosu_total - int(over_work) == 230:
    judgement = True
  #
  #          →    240
  if work == '         ' and kosu_total - int(over_work) == 240:
    judgement = True

#   2
#       1(5)/  2(6)
if tyoku == '5' or tyoku == '6':
  #               →220
  if work == '         ' and kosu_total - int(over_work) == 220:
    judgement = True
  #               →250
  if work == '         ' and kosu_total - int(over_work) == 250:
    judgement = True
```

- `tyoku == '4'`
-
- `tyoku == '5' or '6'`

```
#                                          C  1
#          W  /A  /J/    (W,A)
if (member_obj.shop == 'W1' or member_obj.shop == 'W2' or \
  member_obj.shop == 'A1' or member_obj.shop == 'A2' or \
    member_obj.shop == 'J' or member_obj.shop == '        (W,A)') and \
    #      1
    tyoku == '1':
  #          → 230
  if work == '        ' and kosu_total - int(over_work) == 230:
    judgement = True
  #          → 240
  if work == '        ' and kosu_total - int(over_work) == 240:
    judgement = True
```

- `( ... or ... or ... )` `member_obj.shop`
- `and tyoku == '1'`
-

> ⚠️

```
  #
  if (work == '    ' or work == '        ' or work == '    ' or work == '    ' or work
== '    ' or work == '    ') and \
    #               0      OK
    kosu_total == 0:
    judgement = True

  #         OK/NG
  return judgement
```

- `kosu_total == 0`

- `return judgement` `True` `False` `Business_Time_graph.judgement`

```
kosu_total = 1440 - (      ) - (    45  ) = ... = 470
work=='    '      470 - 0 == 470 → judgement = True  OK
```

`judgement = False`

`judgement_check` `kosu_total` `kosu_total`

> `employee_no5`

```
#
class team_member(models.Model):
  #
  employee_no5 = models.IntegerField('          ')
  #         1
  member1 = models.CharField('                1', max_length=6, blank=True,
null=True)
  #         2
  member2 = models.CharField('                2', max_length=6, blank=True,
null=True)
  #         3
  member3 = models.CharField('                3', max_length=6, blank=True,
null=True)
  #         4
  member4 = models.CharField('                4', max_length=6, blank=True,
null=True)
  #         5
  member5 = models.CharField('                5', max_length=6, blank=True,
null=True)
  #         6
  member6 = models.CharField('                6', max_length=6, blank=True,
null=True)
  #         7
  member7 = models.CharField('                7', max_length=6, blank=True,
null=True)
  #         8
  member8 = models.CharField('                8', max_length=6, blank=True,
null=True)
  #         9
  member9 = models.CharField('                9', max_length=6, blank=True,
null=True)
  #         10
  member10 = models.CharField('               10', max_length=6, blank=True,
null=True)
  #         11
  member11 = models.CharField('               11', max_length=6, blank=True,
null=True)
  #         12
  member12 = models.CharField('               12', max_length=6, blank=True,
null=True)
  #         13
  member13 = models.CharField('               13', max_length=6, blank=True,
null=True)
  #         14
  member14 = models.CharField('               14', max_length=6, blank=True,
null=True)
  #         15
  member15 = models.CharField('               15', max_length=6, blank=True,
```

```
null=True)
  #                    /
  follow = models.BooleanField('        ON/OFF', null=True)

  #
  def __str__(self):
    #
    return str(self.employee_no5)
```

- `employee_no5`
- `member1` `member15` `blank=True, null=True`
- `follow`

> `ForeignKey`

> ⚠️ **`max_length=6`**

`time_work` `A` `x`

>

```
#
class kosu_division(models.Model):
  #              (Ver)
  kosu_name = models.CharField('            Ver  ', blank=True, null=True,
max_length=100)

  # 1
  kosu_title_1 = models.CharField('          1', blank=True, null=True,
max_length=100)
  # 1           (    )
  kosu_division_1_1 = models.TextField('    1', blank=True, null=True)
  # 1
  kosu_division_2_1 = models.TextField('        1', blank=True, null=True)
  # 1
  kosu_division_3_1 = models.BooleanField('    /    1', null=True)

  # 2
  kosu_title_2 = models.CharField('          2', blank=True, null=True,
max_length=100)
  # 2
  kosu_division_1_2 = models.TextField('    2', blank=True, null=True)
  # 2
  kosu_division_2_2 = models.TextField('        2', blank=True, null=True)
  # 2           /
  kosu_division_3_2 = models.BooleanField('    /    2', null=True)
  # …  3        49                        4                                    …
  # 50
  kosu_title_50 = models.CharField('          50', blank=True, null=True,
max_length=100)
  # 50
  kosu_division_1_50 = models.TextField('    50', blank=True, null=True)
  # 50
  kosu_division_2_50 = models.TextField('        50', blank=True, null=True)
  # 50           /
  kosu_division_3_50 = models.BooleanField('    /    50', null=True)

  #
  def __str__(self):
    #   id : Ver
    return str(self.id) + ' : ' + str(self.kosu_name)
```

`kosu_title_N`	

```
kosu_division_1_N
kosu_division_2_N
```

```
#
def kosu_division_dictionary(def_name):
    #
    # Ver        1
    kosu_obj = kosu_division.objects.get(kosu_name=def_name)

    #
    #
    n = 0
    #
    # 1  50
    for kosu_num in range(1, 51):
      # kosu_title_N
      val = getattr(kosu_obj, f'kosu_title_{kosu_num}')
      #
      if val not in [None, '']:
        #
        n = kosu_num

    #
    #
    str_list = list("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwx")[:n]

    #
    # [    ,      ]
    choices_list = []
    #
    for i, m in enumerate(str_list):
      #
      title_val = getattr(kosu_obj, f'kosu_title_{i + 1}')
      # [    ,      ]
      choices_list.append([m, title_val])

    #
    return choices_list, n
```

- `kosu_division.objects.get(kosu_name=def_name)`

- `getattr(kosu_obj, f'kosu_title_{kosu_num}')` `getattr(  , "  ")` `f'kosu_title_{kosu_num}'` `'kosu_title_1'` `'kosu_title_50'`

- `str_list = list("ABC...x")[:n]` `n`

> `.objects.get(...)` `.objects.filter(...)` `.objects.count()`

>

```
#
class def_choice(models.Model):
  #
  def_list = [(x, x) for x in "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwx$"]

  #       A  x,$
  def_symbol = models.CharField('        ', choices=def_list, max_length=2)
  #                   (    )
  def_select = models.CharField('        ', max_length=100)

  #
  def __str__(self):
    #
    return str(self.def_symbol) + str(self.def_select)
```

- `def_list = [(x, x) for x in "ABC...$"]` `"ABC...x$"` `x` `(x, x)` `[('A','A'), ('B','B'), ..., ('$','$')]`
- `def_symbol` `choices=def_list` `max_length=2`
- `def_select`

[for in]

[(x, x) for x in "AB"] [('A','A'),('B','B')]

time_work def_choice A x $ time_work

$

```
#
class administrator_data(models.Model):
  #       1
  menu_row = models.CharField('              ', max_length=4)
  #       1
  administrator_employee_no1 =  models.CharField('                          1',
max_length=255, null=True, blank=True)
  #       2
  administrator_employee_no2 =  models.CharField('                          2',
max_length=255, null=True, blank=True)
  #       3
  administrator_employee_no3 =  models.CharField('                          3',
max_length=255, null=True, blank=True)
  #             1
  pop_up1 = models.CharField('           1', max_length=255, null=True, blank=True)
  #         ID1
  pop_up_id1 =  models.CharField('            ID1', max_length=255, null=True,
blank=True)
  #             2
  pop_up2 = models.CharField('           2', max_length=255, null=True, blank=True)
  #         ID2
  pop_up_id2 =  models.CharField('            ID2', max_length=255, null=True,
blank=True)
  #             3
  pop_up3 = models.CharField('           3', max_length=255, null=True, blank=True)
  #         ID3
  pop_up_id3 =  models.CharField('            ID3', max_length=255, null=True,
blank=True)
  #             4
  pop_up4 = models.CharField('           4', max_length=255, null=True, blank=True)
  #         ID4
  pop_up_id4 =  models.CharField('            ID4', max_length=255, null=True,
blank=True)
  #             5
  pop_up5 = models.CharField('           5', max_length=255, null=True, blank=True)
  #         ID5
  pop_up_id5 =  models.CharField('            ID5', max_length=255, null=True,
blank=True)

  #
  def __str__(self):
    #       1
    return '    ' + str(self.id)
```

- `menu_row`

- `administrator_employee_no1 3` `inquiry_data`
- `pop_up1 5` `pop_up_id1 5` `member`

>

>

```
#
class inquiry_data(models.Model):
  #
  content_list = [
    #
    ('    ', '    '),
    #
    ('      ', '      '),
    #
    ('          ' ,'          '),
  ]

  #
  employee_no2 = models.IntegerField('          ')
  #
  name = models.ForeignKey(member, verbose_name='    ', null=True,
on_delete=models.SET_NULL)
  #
  content_choice = models.CharField('        ', choices=content_list, max_length=5)
  #          OK
  inquiry = models.TextField('          ')
  #
  answer = models.TextField('    ', null=True, blank=True)
  #
  created_at = models.DateTimeField(auto_now_add=True)
  #
  updated_at = models.DateTimeField(auto_now=True)

  #
  def __str__(self):
    #   id
    return str(self.id) + str(self.name)
```

- `content_choice`
- `inquiry` `TextField`
- `answer`
- `created_at = models.DateTimeField(auto_now_add=True)` `auto_now_add=True`
- `updated_at = models.DateTimeField(auto_now=True)` `auto_now=True`

> **`auto_now_add`** **`auto_now`** ⚠️

> - `auto_now_add=True`
> - `auto_now=True` `name`
>
>   `on_delete=SET_NULL`

`AsyncTask` `History`

>

```python
#
class AsyncTask(models.Model):
  #
  MAX_RECORDS = 1000

  #       ID
  task_id = models.CharField(max_length=255, unique=True)
  #
  status = models.CharField(max_length=50, choices=[
    #
    ('pending', 'Pending'),
    #
    ('success', 'Success'),
    #
    ('error', 'Error')
  ])
  #
  result = models.TextField(null=True, blank=True)
  #
  created_at = models.DateTimeField(auto_now_add=True)
  #
  updated_at = models.DateTimeField(auto_now=True)

  #
  def __str__(self):
    return f'{self.created_at} on {self.status} (TaskID: {self.task_id})'

  #
  def save(self, *args, **kwargs):
    #
    super().save(*args, **kwargs)
    #
    current_count = self.__class__.objects.count()

    #
    #     (1000)
    if current_count > self.MAX_RECORDS:
      #
      #
      excess_count = current_count - self.MAX_RECORDS
      #
      oldest_records = self.__class__.objects.order_by('created_at')[:excess_count]
      #
      for record in oldest_records:
        # 1
        record.delete()
```

- `task_id = models.CharField(..., unique=True)` `unique=True`

- `status` `pending` `success` `error`
- `def save(self, *args, **kwargs):` **`save`**

- `super().save(*args, **kwargs)` `super()` `models.Model`

- `current_count = self.__class__.objects.count()` `self.__class__` `.objects.count()`
- `if current_count > self.MAX_RECORDS:`
- `oldest_records = ...order_by('created_at')[:excess_count]` `order_by('created_at')`

- `for record in oldest_records: record.delete()`

> `AsyncTask` `History`

> ⚠️ **`save()`**
>
> `MAX_RECORDS`

```
#
class Operation_history(models.Model):
  #
  created_at = models.DateTimeField(auto_now_add=True)
  #
  employee_no4 = models.IntegerField('          ')
  #
  name = models.ForeignKey(member, verbose_name='    ', null=True,
on_delete=models.SET_NULL)
  #
  post_page = models.CharField('      ', max_length=255, null=True, blank=True)
  #
  operation_models = models.CharField('             ', max_length=255, null=True,
blank=True)
  #     /
  status = models.CharField('    ', max_length=255, null=True, blank=True)
  #
  operation_detail = models.TextField('        ', null=True, blank=True)

  #
  def __str__(self):
    return str(self.id) + str(self.name) + '  ' + str(self.created_at)
```

-

- `name` `on_delete=SET_NULL`

>

```python
#
class History(models.Model):
    #           50
    MAX_RECORDS = 500000

    #           create/update/delete
    operation = models.CharField(max_length=10)
    #
    table_name = models.CharField(max_length=50)
    #               id
    record_id = models.IntegerField()
    #
    login_No = models.CharField(max_length=255, blank=True, null=True)
    #             JSON
    changes = jsonfield.JSONField(blank=True, null=True)
    #
    timestamp = models.DateTimeField(auto_now_add=True)

    #
    def __str__(self):
        return f'{self.operation} on {self.table_name} (ID: {self.record_id})'

    #                     AsyncTask
    def save(self, *args, **kwargs):
        #
        super().save(*args, **kwargs)
        #
        current_count = self.__class__.objects.count()

        #
        # 50
        if current_count > self.MAX_RECORDS:
            #
            #
            excess_count = current_count - self.MAX_RECORDS
            #
            oldest_records = self.__class__.objects.order_by('timestamp')[:excess_count]
            #
            for record in oldest_records:
                #
                record.delete()
```

- `changes = jsonfield.JSONField(...)`

  `{"name": ["  ", "  "]}` `CharField`

- `operation` `table_name` `record_id` `login_No`

- `save` `AsyncTask` `timestamp`

> `{ "  ": "  " }`

> **`Operation_history`** **`History`** `Operation_history`
> `History`
>
> `signals.py`

`shop_list`	`judgement_check` `kosu_sort` `break_get`
`time_work` `#` `$`	
`#HHMMHHMM`	`break_time_process`
`judgement_check`	
`kosu_division`	`getattr(obj, f'kosu_title_{n}')`
`member` `break_time1` `6`	`break_get` `tyoku`

`models.py`

```
#
python manage.py makemigrations
#
python manage.py migrate
```

> ⚠️ **`makemigrations`** **`migrate`**

`on_delete=SET_NULL`

`Business_Time_graph` `tasks.py`

`employee_no3` `employee_no2` `employee_no4`
`employee_no5` `name` `NULL`

`AsyncTask` `History`

---

judgement=False	judgement_check kosu_total	$ #
	break_next_day	break_time_process
		Business_Time_graph.def_ver2 kosu_division.kosu_name
	SET_NULL PROTECT	on_delete
	SET_NULL	employee_no
	AsyncTask History	
time_work	$	
		makemigrations migrate
	detail_list_summarize $	$
	member.break_check	break_check
	break_get tyoku tyoku2	Business_Time_graph.tyoku2

---

time_work max_length=288

`#09001000`

`#09001000` `break_time_process` `break_start` `break_end`
`HH×12 + MM÷5`

`break_time_delete` `member.break_check` `True` `False`

`tyoku='4'` `judgement` `True`
`kosu_total`

`on_delete=models.SET_NULL` `models.CASCADE`
`SET_NULL`

`kosu_division`

`AsyncTask` `save` `MAX_RECORDS`

`detail_list_summarize` `$`
`split('$')`

`kosu_division_dictionary` `break_get`

- `makemigrations` `migrate`
- `member` `Business_Time_graph`
  `inquiry_data` `Operation_history` `ForeignKey` `member`
- **`on_delete=SET_NULL`** `employee_no`
- **`time_work`**
  `#` `$` `HH×12 + MM÷5`

- `#HHMMHHMM` `#` `break_time_process`
- `kosu_utils.py` `kosu_write` `break_time_delete` `break_time_write` `detail_list_summarize` `judgement_check`
- `judgement_check`
- `member` `break_get` `tyoku`
- `kosu_division`
- `AsyncTask` `History` `save`
- `time_work`